ข้อที่ 1: Array & Function (Divisible Pair Positions)

โจทย์: จงเขียนโปรแกรมเปรียบเทียบอาร์เรย์ 2 ตัว (A และ B) ที่มีขนาดเท่ากัน เพื่อหาว่ามีกี่ตำแหน่งที่ "สมาชิกในอาร์เรย์ A สามารถหารด้วยสมาชิกในอาร์เรย์ B ที่ตำแหน่งเดียวกันลงตัวพอดี" (A[i] % B[i] == 0) โดยมีเงื่อนไขว่าตัวหารในอาร์เรย์ B ต้องไม่เป็น 0

กำหนด Function Prototype ให้ดังนี้: `int countDivisible(int A[], int B[], int length);`

กำหนดให้รับอาร์เรย์ A และ B ขนาด `length` เข้าไปตรวจสอบ ฟังก์ชันนี้ต้อง **คืนค่า (Return)** เป็นจำนวนคู่ที่เข้าเงื่อนไข และให้ทำการ **Print ตำแหน่ง Index (เริ่มนับที่ 1)** ของคู่ที่เข้าเงื่อนไขออกมาจากในฟังก์ชันด้วย (หรือจัดการตามความเหมาะสมของโค้ดคุณ)

Input: บรรทัดแรก: ขนาดของอาร์เรย์ (N) บรรทัดที่สอง: สมาชิกของอาร์เรย์ A จำนวน N ตัว บรรทัดที่สาม: สมาชิกของอาร์เรย์ B จำนวน N ตัว

Output: บรรทัดแรก: แสดงตำแหน่ง (Index ที่เริ่มจาก 1) ของคู่ที่หารลงตัว คั่นด้วยช่องว่าง บรรทัดที่สอง: แสดงจำนวนคู่ทั้งหมดที่เข้าเงื่อนไข

ตัวอย่าง Input	ตัวอย่าง Output
5	1 3 5
10 15 20 8 13	3
2 4 5 0 13	

ตำแหน่งที่ 1 คือ 10/2 ลงตัว, ตำแหน่งที่ 3 คือ 20/5 ลงตัว, ตำแหน่งที่ 5 คือ 13/13 ลงตัว. ส่วนตำแหน่งที่ 2 คือ 15/4 ไม่ลงตัว และตำแหน่งที่ 4 ข้ามไปเพราะ B เป็น 0

ข้อที่ 2: String Manipulation (Secure Hex Code)

โจทย์: ระบบต้องการกรองรหัสสี Hex Code จากข้อมูลที่ผู้ใช้กรอกเข้ามา จงเขียนโปรแกรมรับค่า String N ตัว และแสดงผลเฉพาะ String ที่เป็น Hex Code ที่ถูกต้องตามเงื่อนไขดังนี้:

1. ต้องมีความยาว **7 ตัวอักษรพอดี**
2. อักขระตัวแรกต้องเป็นเครื่องหมาย `#` เสมอ
3. อักขระที่เหลืออีก 6 ตัว ต้องประกอบไปด้วยตัวเลข `0-9` หรือตัวอักษรภาษาอังกฤษ `A-F` หรือ `a-f` เท่านั้น
4. **เงื่อนไขพิเศษ:** ใน 6 ตัวหลัง ต้องมีตัวอักษรภาษาอังกฤษ (A-F หรือ a-f) ซ่อนอยู่อย่างน้อย 1 ตัว (ห้ามเป็นตัวเลขล้วน)

Input: บรรทัดแรก: จำนวน String ที่ต้องการตรวจสอบ (N) N บรรทัดถัดมา: String แต่ละบรรทัด

Output: พิมพ์เฉพาะ String ที่ตรงตามเงื่อนไขบรรทัดละ 1 ค่า หากไม่มีข้อใดถูกเลยไม่ต้องพิมพ์อะไร

ตัวอย่าง Input	ตัวอย่าง Output
5	#A2c4E9
#123456	
#A2c4E9	
#9999Z9	
hello	
#12b45	

#123456 ผิดเพราะไม่มีตัวอักษรเลย, #9999Z9 ผิดเพราะมี Z, hello ผิดฟอร์แมต, #12b45 ผิดเพราะยาวไม่ถึง 7

ข้อที่ 3: Struct (Top Salesperson)

โจทย์: จงเขียนโปรแกรมเก็บข้อมูลพนักงานขายจำนวน N คน โดยใช้ `struct` แต่ละคนจะประกอบด้วย ชื่อ (String), นามสกุล (String), และยอดขายในแต่ละไตรมาส (Array ขนาด 4 ช่อง เก็บเป็นตัวเลขทศนิยม float)

กำหนด `struct` และ Function Prototype:

```
typedef struct {
    char name[30];
    char surname[30];
    float sales[4];
    float totalSales;
} Salesperson;


void calculateTotal(Salesperson *s);
void printTopSeller(Salesperson emp[], int size);
```

Input: บรรทัดแรก: จำนวนพนักงาน (N) N บรรทัดถัดมา: ข้อมูลพนักงานแต่ละคน (ชื่อ นามสกุล ยอดขายQ1 ยอดขายQ2 ยอดขายQ3 ยอดขายQ4) คั่นด้วยช่องว่าง

Output: แสดงชื่อ นามสกุล และยอดขายรวม (ทศนิยม 2 ตำแหน่ง) ของพนักงานที่มียอดขายรวม **สูงที่สุด** เพียงคนเดียว (รับประกันว่าไม่มีคนได้ยอดรวมเท่ากัน) เรียกใช้ผ่าน `printTopSeller()`

ตัวอย่าง Input	ตัวอย่าง Output
3 John Doe 10.5 20.0 15.0 10.0 Jane Smith 25.0 25.0 20.5 30.0 Bob Marly 5.0 5.0 5.0 5.0	Jane Smith 100.50

ข้อที่ 4: Sequence (Jump Sequence)
โจทย์:

กำหนดลำดับตัวเลขของอนุกรมให้ดังนี้

2, 5, 14, 41, 122, 365, …

จงหาความสัมพันธ์ของอนุกรมนี้ และเขียนโปรแกรมเพื่อหาค่าในพจน์ที่ n

Input:

พจน์ที่ n (1 <= n <= 20)

Output:

ค่าของพจน์ที่ n

ตัวอย่าง Input	ตัวอย่าง Output
2	5
5	122